



Departamento de Ciencias de la Computación

Carrera de Ingeniería de Software

Desarrollo de aplicaciones móviles

Examen Práctico 2P

Tema: Sistema de gestión documental inteligente.

Integrantes:

ALEXIS SANTIAGO CHIMBA PORTERO

SEGUNDO JAVIER RAMOS YANCHALIKUIN

ARIEL LEONIDAS REYES CERON

ANTHONY NESTOR VILLARREAL MACIAS

Docente: DORIS KARINA CHICAIZA ANGAMARCA

Fecha: 08/05/2025

Gestión Documental Inteligente

Caso práctico:

Una institución pública implementa un sistema que almacena miles de documentos normativos: leyes, reglamentos internos, resoluciones y manuales. Este sistema usa embeddings con all-MiniLM-L6-v2 y ChromaDB o FAISS para búsquedas semánticas.

¿Cómo funciona?

- El usuario (ciudadano o funcionario) ingresa:
 - “¿Qué normativa regula los contratos de obras menores?”
- El sistema transforma esa pregunta en un vector y recupera los documentos más cercanos semánticamente, aunque no contengan exactamente esas palabras.
- Además muestra resúmenes automáticos generados por el LLM.

Beneficios prácticos:

- Facilita el cumplimiento normativo, evitando multas o retrasos.
- Acelera el trabajo jurídico o administrativo, ya que encuentra regulaciones afines aunque la búsqueda no sea exacta.
- Integra control de versiones y auditoría (saber quién consultó o actualizó documentos).

Documentación del Sistema de Gestión Documental Inteligente

1. Descripción General

El Sistema de Gestión Documental Inteligente es una solución tecnológica que permite a instituciones públicas gestionar, buscar y analizar documentos normativos mediante técnicas avanzadas de procesamiento de lenguaje natural y búsqueda semántica.

2. Arquitectura del Sistema

a. Diagrama de Arquitectura

SISTEMA COMPLETO		
BACKEND (Google Colab/Flask)	FRONTEND (Aplicación Android)	MODELOS IA
- Servidor Flask	- Interfaz de usuario	- Modelo de embeddings
- ChromaDB/FAISS	- Gestión de estado	(all-MiniLM-L6-v2)
- Base de datos SQLite	- Llamadas API	- Modelo de resumen
- Ngrok para tunneling	- Historial de búsquedas	(facebook/bart-large-cnn)

b. Componentes Principales

Backend (Google Colab/Flask)

- **Servidor Flask:** Maneja las solicitudes HTTP y las respuestas JSON

- **ChromaDB:** Almacena los embeddings de los documentos para búsqueda semántica
- **SQLite:** Base de datos para registro de auditoría y control de versiones
- **Ngrok:** Proporciona un túnel seguro para exponer el servidor local a internet

Frontend (Aplicación Android)

- **Interfaz de usuario:** Diseñada con Flutter para una experiencia responsive
- **Gestión de estado:** Usa Provider para manejar el estado de la aplicación
- **Almacenamiento local:** SQLite para el historial de búsquedas

Modelos de IA

- **Sentence Transformer (all-MiniLM-L6-v2):** Genera embeddings para búsqueda semántica
- **BART Large CNN:** Modelo de resumen automático de textos

3. Flujo de Trabajo

a. Proceso de Búsqueda

- **Consulta del usuario:** El usuario ingresa una pregunta en lenguaje natural
- **Transformación a vector:** El texto se convierte en un embedding usando all-MiniLM-L6-v2
- **Búsqueda semántica:** ChromaDB encuentra los documentos más cercanos en el espacio vectorial
- **Generación de resúmenes:** El modelo BART genera resúmenes concisos de los documentos encontrados
- **Respuesta:** Los resultados se envían al cliente con los documentos relevantes y sus resúmenes

b. Proceso de Auditoría

- **Registro de consulta:** Cada búsqueda se registra en la base de datos SQLite
- **Control de versiones:** Cada documento tiene un historial de versiones
- **Identificación de usuario:** Se registra el ID del usuario que realiza la consulta

4. Implementación Técnica

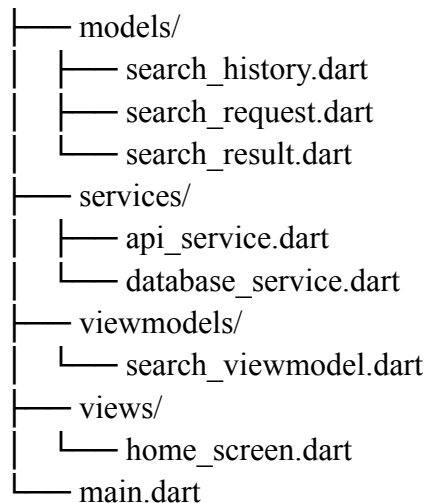
a. Backend

GoogleColab

b. Frontend (Android/Flutter)

Estructura de directorios

lib/



5. Consideraciones de Despliegue

a. Backend

- **Requisitos:** Python 3.7+, 4GB+ RAM (para modelos de embeddings)
- **Dependencias:**
 - Flask, ChromaDB, SentenceTransformers, Transformers
- **Escalabilidad:**
 - Para producción, considerar despliegue en instancia cloud con GPU
 - Reemplazar Ngrok con solución más estable (AWS ALB, GCP Load Balancer)

b. Frontend

- **Requisitos:** Android 8.0+ (API 26)
- **Dependencias:**
 - Flutter 3.0+, http, provider, sqflite
- **Optimizaciones:**
 - Cachear resultados frecuentes
 - Implementar paginación para muchos resultados

6. Mejoras Futuras

- **Indexación incremental:** Actualizar embeddings cuando se añadan nuevos documentos
- **Búsqueda facetada:** Filtrar por tipo de documento (ley, reglamento, etc.)
- **Sistema de feedback:** Permitir a usuarios valorar la relevancia de resultados
- **Notificaciones:** Alertar sobre cambios en documentos consultados previamente
- **Análisis de tendencias:** Identificar consultas frecuentes para optimización

7. Consideraciones de Seguridad

- **Autenticación:** Implementar JWT para acceso a la API
- **Cifrado:** Usar HTTPS para todas las comunicaciones
- **Control de acceso:** Restringir ciertos documentos por roles de usuario
- **Auditoría:** Mantener registro detallado de accesos y modificaciones
- **Protección de datos:** Anonimizar datos personales en los registros

8. Conclusión

Este sistema proporciona una solución completa para la gestión documental inteligente en instituciones públicas, combinando búsqueda semántica, resumen automático y auditoría detallada. La arquitectura modular permite fácil expansión y adaptación a diferentes tipos de documentos y flujos de trabajo.